Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Session	**2025-26 (ODD)**	**Course Name**	**Operating System Lab**
Semester	**5**	**Course Code**	**23IOT1504**
Roll No	**64**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**07**
Course Outcome	1. Understand Computer System Configuration and Simulate system resources efficiently using Linux Commands (CO1) 2. Analyse operating system functionalities utilizing system calls, thread programming and process scheduling algorithms (CO2) 3. Apply Synchronization primitives to implement a Deadlock-free solution(CO3) 4. Simulate Disk scheduling, Memory allocation, File allocation, page replacement algorithms (CO4)
Aim	Implement a program to simulate disk scheduling algorithms. A. FCFS B. SSTF
Theory	In computer systems, a disk scheduling algorithm determines the order in which pending disk I/O (input/output) requests are served. Since a hard disk drive consists of moving parts such as a rotating platter and a read/write head, the time taken to access data (known as seek time) depends on the position of the head. Efficient disk scheduling helps minimize seek time and improve system performance. A. FCFS: B. SSTF ❖ **First Come First Serve (FCFS):** The First Come First Serve (FCFS) algorithm is the simplest disk scheduling technique. It processes requests in the exact order they arrive in the queue, without reordering them based on location. The disk head moves sequentially to each requested track as per arrival order. Although this method is fair and easy to implement, it often leads to large head movements and higher average seek time, especially when the requests are scattered across the disk. For example, if the head starts at track 50 and the queue is 82, 170, 43, 140, 24, 16, and 190, the head will move through them in that same order, resulting in inefficient performance. ❖ **Shortest Seek Time First (SSTF):** The Shortest Seek Time First (SSTF) algorithm improves upon FCFS by selecting the request closest to the current head position. It calculates the seek distance for each pending request and services the one that requires

	the least movement. This approach significantly reduces total head movement and average seek time compared to FCFS. However, a disadvantage of SSTF is that requests far from the current head position may starve, as nearer requests are continuously prioritized. For example, with the same request sequence and head starting at 50, SSTF will serve requests in the order 43, 24, 16, 82, 140, 170, and 190, resulting in shorter total head movement.

Procedure and Execution	**A. FCFS:**
	Code:
	//sheikhwasimuddin 23071044 CSEIOT-64
	#include <stdio.h>
	#include <stdlib.h>
	#include <math.h>
	void FCFS(int requests[], int n, int head) {
	int total_movement = 0;
	printf("\nFCFS Disk Scheduling:\n");
	printf("Head movement order: %d", head);
	for (int i = 0; i < n; i++) {
	total_movement += abs(requests[i] - head);
	head = requests[i];
	printf(" -> %d", head);
	}
	printf("\nTotal Head Movement = %d\n", total_movement);
	printf("Average Head Movement = %.2f\n", (float)total_movement / n);
	}
	int main() {
	int n, head;
	printf("Enter number of disk requests: ");
	scanf("%d", &n);
	
	int requests[n];
	printf("Enter the disk request sequence:\n");
	for (int i = 0; i < n; i++)
	scanf("%d", &requests[i]);
	printf("Enter initial head position: ");
	scanf("%d", &head);
	FCFS(requests, n, head);
	return 0;
	}

B. **SSTF:**

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Function for SSTF Disk Scheduling
void SSTF(int requests[], int n, int head) {
    int total_movement = 0;
    int completed = 0;
    int visited[n];

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("\nSSTF Disk Scheduling:\n");
    printf("Head movement order: %d", head);

    while (completed < n) {
        int min_distance = 9999;
        int index = -1;

        // Find the closest unvisited request
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                int distance = abs(requests[i] - head);
                if (distance < min_distance) {
                    min_distance = distance;
                    index = i;
                }
            }
        }

        visited[index] = 1;
        total_movement += min_distance;
        head = requests[index];
        completed++;

        printf(" -> %d", head);
    }

    printf("\nTotal Head Movement = %d\n", total_movement);
    printf("Average Head Movement = %.2f\n", (float)total_movement / n);
}
```

```
int main() {
    int n, head;

    printf("Enter number of disk requests: ");
    scanf("%d", &n);

    int requests[n];
    printf("Enter the disk request sequence:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &requests[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    SSTF(requests, n, head);

    return 0;
}
```

Output:

FCFS

main.c

```c
//sheikhwasimuddin 23071044 CSEIOT-64
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

void FCFS(int requests[], int n, int head) {
    int total_movement = 0;

    printf("\nFCFS Disk Scheduling:\n");
    printf("Head movement order: %d", head);

    for (int i = 0; i < n; i++) {
        total_movement += abs(requests[i] - head);
        head = requests[i];
        printf(" -> %d", head);
    }

    printf("\nTotal Head Movement = %d\n", total_movement);
    printf("Average Head Movement = %.2f\n", (float)total_movement / n)
}

int main() {
```

```
Enter number of disk requests: 7
Enter the disk request sequence:
82 170 43 140 24 16 190
Enter initial head position: 50

FCFS Disk Scheduling:
Head movement order: 50 -> 82 -> 170 -> 43 -> 140 -> 24 -> 16 -> 190
Total Head Movement = 642
Average Head Movement = 91.71


...Program finished with exit code 0
Press ENTER to exit console.
```

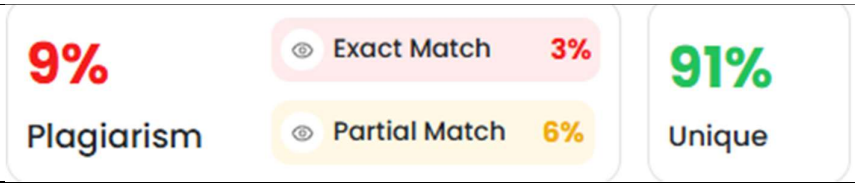
SSTF

main.c

```c
//sheikhwasimuddin 23071044 CSEIOT-64
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

void SSTF(int requests[], int n, int head) {
    int total_movement = 0;
    int completed = 0;
    int visited[n];

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("\nSSTF Disk Scheduling:\n");
    printf("Head movement order: %d", head);

    while (completed < n) {
        int min_distance = 9999;
        int index = -1;
```

```
Enter number of disk requests: 7
Enter the disk request sequence:
82 170 43 140 24 16 190
Enter initial head position: 50

SSTF Disk Scheduling:
Head movement order: 50 -> 43 -> 24 -> 16 -> 82 -> 140 -> 170 -> 190
Total Head Movement = 208
Average Head Movement = 29.71


...Program finished with exit code 0
Press ENTER to exit console.
```

Output Analysis	In the FCFS (First Come First Serve) scheduling, the disk head processes the requests in the exact order they arrive. For the input sequence {82, 170, 43, 140, 24, 16, 190} with the initial head at 50, the head moves sequentially to each request in order. The total head movement is the sum of all distances moved, and since it does not choose the shortest path, it results in a larger total head movement.In the SSTF (Shortest Seek Time First) scheduling, the disk head always serves the request that is closest to the current head position, minimizing each movement. For the same request sequence and head at 50, the algorithm selects the nearest track at every step. This leads to less total head movement compared to FCFS.Overall, the SSTF algorithm improves performance by reducing average seek time and total head movement, while FCFS is simpler but less efficient when requests are scattered far apart on the disk.
Conclusion	The FCFS and SSTF disk scheduling algorithms both manage how the disk head services I/O requests, but they differ in efficiency. **FCFS** is simple and easy to implement, serving requests in the order they arrive but may cause more head movement and longer access times. **SSTF**, on the other hand, selects the nearest request first, resulting in **reduced total head movement and faster performance**. Hence, **SSTF is more efficient** than FCFS in terms of seek time and overall disk performance.

Plag Report (Similarity index < 12%)	
Date	27/10/2025